Exact Evaluation of Time-Series Anomaly Metrics by Local Event Statistics

Youngmin Ko
rhdudals0505@naver.com
Independent Researcher
Republic of Korea

ABSTRACT

Time-series anomaly evaluation repeatedly scans score thresholds to compute range-aware metrics. We derive local event statistics for two existing operators without changing their definitions. For dynamic VUS, inactive PR events vanish while ROC requires a predecessor correction; complete global ties and interval geometry give a support-sensitive exact representation. For flat reciprocal-cardinality range precision/recall, endpoint activations update both overlap and fragmentation statistics in constant time per inserted point. Exhaustive observed-threshold evaluation takes $O(n \log n + n)$, while H ordered queries admit $O(n \log(H+1) + n + H)$ through ordinary offline bucketing. A frozen study on 73 real series and eight score procedures, including both operators, is reproduced on two CPU architectures. A separate range-metric study on the same 584 profiles compares a fused compiled raster with numerically matching outputs: exhaustive range-F1 evaluation improves geometrically by 27.8×, 21.4× and 128.1× on NAB, SMAP and SMD; fixed 100-query evaluation improves 1.72–1.99×. The 100-query approximation selects nonoverlapping winning models in 15/73 cases, twelve with range-F1 regret at least .01. Whole-consumer significance remains bounded: accelerating VUS alone in a public nine-metric workload gives only 1.031× overall. We retain conventional-control wins, low-cardinality losses, source dependence and numerical limits. Classical component trees, sorting and caching are prior techniques; the contribution is the operator-specific exact state representation and its measured evaluation consequences.

PVLDB Reference Format:
Youngmin Ko. Exact Evaluation of Time-Series Anomaly Metrics by Local Event Statistics. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/mandu5/sparse-vus/tree/research/pvldb-20261024.

1 INTRODUCTION

Evaluation is a repeated data-analysis operation in time-series anomaly benchmarks: stored score vectors from several detectors or configurations are joined with the same labels, evaluated, and compared to select a numerical winner. Volume Under the Surface (VUS) aggregates range-aware precision–recall (PR) or receiver-operating-characteristic (ROC) integrals over temporal widths [2, 18]. Exhaustive evaluation integrates every complete distinct-score threshold at every integer width. Sampling thresholds reduces work but changes the returned scalar and can change the selected model. The computational question is whether exact aggregates need repeated complete threshold scans. We study both dynamic VUS and the existing flat reciprocal-cardinality range precision/recall metric. The latter cannot discard off-label events; instead, it admits local updates as predicted intervals merge.

Substantial acceleration already exists. Boniol et al. separate maximum-buffer support from unaffected regions and reuse static work. Klein's fast-VUS uses support-score cumulative sums; Benchopt uses dense cumulative sums and segment maxima [1, 2, 8]. However, reusing support statistics still leaves a second question: which threshold states must be retained to compute the scalar integral? PR and ROC give different answers, because a threshold with no new true-positive mass can still contribute ROC area.

We derive a scalar event representation of the original dynamic operator. Inactive threshold runs have zero PR area, and their ROC area is a horizontal rectangle preceding the next active transition. This distinction matters: connecting only retained ROC endpoints gives an incorrect answer, even without buffers. Complete global score ties must include predictions outside support, and expanded anomaly components must merge under the target's overlap rule. A two-distance lemma removes repeated interval-ramp construction. With standard binary64 radix preparation, exhaustive evaluation takes $O(n + (L+1)Q)$, where n is series length, L is maximum width, and Q is maximum-width support. This is a support-sensitive bound, not a claim that sorting arbitrary real numbers is linear.

The data-management contribution is an exact implementation of a recurring analytics operator on stored detector profiles. It reduces the states and width-dependent work needed for the same aggregate, and exposes shared-label preparation when several score columns are evaluated together. The latter uses standard caching and loop interchange; its value is measured separately from the event representation. Our complete workflow reads stored arrays, evaluates all score columns, selects winners, and writes the decision table. We do not claim a new detector, deployment selector, metric, or database execution engine.

A preliminary evaluator and working manuscript were publicly released as Sparse VUS 0.3.0 on 21 October 2026 [10]. This paper presents the complete exact-evaluator work and separately measures the sharing increment over that unrefereed software precursor.

The second operator needs a different event representation: predicted intervals merge even outside true ranges, and fragmentation

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

makes recall nonmonotone. For flat reciprocal-cardinality range precision/recall, we maintain exact overlap and component-count contributions in constant time per activation. Endpoint arrays specialize classical connected-level-set processing to a path; sorting and connectivity themselves are prior techniques. An exhaustive observed-score query therefore takes $O(n \log n + n)$ instead of repeated $O(n)$ scans. For H supplied ordered queries, offline buckets give $O(n \log(H+1)+n+H)$ including preparation. This new range representation was not in the public VUS precursor.

Together, the developments make three connected contributions. First, they prove the support-event identities and their complexity, including the dynamic denominator, complete ties and ROC predecessor correction, and the distinct local-statistics identity for range precision/recall. Second, they provide implementations and ablations separating shared preparation, compiled raster recomputation, sorting and query bucketing. Third, they evaluate full public calls and complete collection workflows on a prospectively frozen, full-precision panel, followed by separately frozen comparator, approximation and range-generalization studies. The experiments report confidence intervals, label-block sensitivity, exact-value checks and unsuccessful alternatives. The primary study uses three public univariate collections and eight fixed score-generation procedures, with fresh execution on two CPU architectures. Record reuse and the scope of warm stored-score evaluation are stated with the experimental protocol.

2 TARGET AND EXACT EVENT REPRESENTATION

Let $y \in \{0,1\}^n$ contain both classes and $s \in \mathbb{R}^n$ contain finite scores after binary64 conversion. Larger scores indicate greater anomaly evidence. Let $P_0 = \sum_i y_i$, and let the K maximal original anomaly intervals have inclusive endpoints $[a_k, b_k]$ in temporal order. Widths are the integers $w = 0, \dots, L$. The continuous label $a_i^{(w)}$ is one on original positives. An interval contributes $\sqrt{1 - d/w}$ outside its endpoints at integer distance $1 \le d \le \lfloor w/2 \rfloor$; contributions are summed and clipped at one. At $w = 0$ no ramp is added. Expanded intervals that overlap merge, while adjacent nonoverlapping intervals remain separate. This is the original dynamic VUS target pinned to the official `RangeAUC_volume_opt` source at revision `467bace0`, configured for exhaustive thresholds [25]. It is not shared by all software named VUS.

For a threshold t, define

$$p_i(t) = \mathbf{1}\{s_i \ge t\}, \qquad C(t) = \sum_i p_i(t), \tag{1}$$

$$A(t) = \sum_i y_i p_i(t), \qquad B(t) = \sum_i (1 - y_i) a_i^{(w)} p_i(t), \tag{2}$$

$$T(t) = A(t) + B(t), \qquad P(t) = P_0 + B(t)/2, \tag{3}$$

$$N(t) = n - P(t). \tag{4}$$

Let $H(t)$ count expanded components hit by at least one predicted positive, and let K_w count those components. The curve coordinates are

$$R(t) = \min\{T(t)/P(t), 1\}\, H(t)/K_w, \tag{5}$$

$$F(t) = (C(t) - T(t))/N(t), \qquad \pi(t) = T(t)/C(t). \tag{6}$$

Precision is used only at nonempty retained thresholds, where $C(t) > 0$; the initial origin requires no precision value. The denominator is positive: $P(t) \le (n + P_0)/2 < n$. ROC integrates trapezoids from the origin through complete distinct descending score thresholds to the explicit terminal $(1, 1)$. PR uses the right-step sum $\sum_j (R_j - R_{j-1})\pi_j$, with $R_0 = 0$. Each VUS scalar is the arithmetic mean over every width. Signed horizontal ROC increments are preserved; the derivation does not replace a dynamic denominator by a fixed one.

Let $S_w = \{i : a_i^{(w)} > 0\}$, $Q_w = |S_w|$, and $Q = Q_L$. For an active event with score v_j, precompute the complete global counts $c_j^{>} = |\{i : s_i > v_j\}|$ and $c_j^{\ge} = |\{i : s_i \ge v_j\}|$. They include positions outside S_w and outside S_L.

Lemma 1 (Inactive threshold runs). *Between consecutive active score events at a fixed width, T, B, P, N, H, R are constant. Omitted thresholds contribute a horizontal ROC rectangle and zero PR area. At an active score, its complete global numerical tie is a single transition.*

Proof. A newly admitted inactive point has zero weight and cannot hit an expanded component. Only C changes. The horizontal increments $(\Delta C)/N$ telescope at fixed height R. All positions tied with an active event enter at its global threshold, so both counts must retain them. □

If the previous retained state is (T_o, N_o, R_o, c_o), event j contributes

$$f_j^{-} = (c_j^{>} - T_o)/N_o, \tag{7}$$

$$\Delta\mathrm{ROC}_j = (c_j^{>} - c_o)R_o/N_o + (F_j - f_j^{-})(R_j + R_o)/2, \tag{8}$$

$$\Delta\mathrm{PR}_j = (R_j - R_o)T_j/c_j^{\ge}. \tag{9}$$

The initial state is $(0, n - P_0, 0, 0)$, and afterwards $c_o = c_j^{\ge}$. Connecting retained endpoints without the intervening rectangle is incorrect. For example, $y = (1, 0, 1, 0)$, $s = (3, 2, 1, 0)$ and $L = 0$ give ROC area 0.625; connecting support endpoints alone gives 0.8125.

Lemma 2 (Two-distance geometry). *The weight of a nonoriginal point is determined by its distances d_1, d_2 to the nearest two distinct original intervals. It is zero if $d_1 > \lfloor w/2 \rfloor$, one if $d_2 \le \lfloor w/2 \rfloor$, and $\sqrt{1 - d_1/w}$ otherwise.*

Proof. Every active ramp has weight at least $1/\sqrt{2}$. Two contributions therefore clip to one. If exactly one interval contributes, its distance determines the weight. Distances to ordered intervals are monotone moving away on each side, so two neighbors on each side contain the nearest two intervals. □

Theorem 1 (Support-event evaluation). *In a unit-cost model with score comparisons and metric arithmetic, including square roots, both exhaustive scalar integrals can be computed in $O(n \log(Q+1) + (L+1)Q)$ time and $O(n + Q)$ auxiliary words.*

Proof. Extract original intervals and maximum support in $O(n)$ time. Sort the Q support scores and group equal values. Binary-search all n scores among these groups to obtain complete global counts $c^{>}, c^{\ge}$ without sorting off-support scores, at cost $O(n \log(Q+1))$. The ordered-interval geometry can be prepared

within this bound. At each width, compute support weights, component membership, grouped masses and first component-hit events in $O(Q + K)$ time. A scan of the at most Q event groups integrates the preceding rectangles, complete tied transitions and terminal ROC tail. The inactive-run lemma makes this identical to the exhaustive target in exact metric arithmetic. Stream the width mean using input-, support- and interval-sized arrays. Since $K \le Q \le n$, the stated bounds follow. □

Proposition 1 (Fixed-binary64 preparation). *For finite binary64 scores, complete global tie counts, the score ordering of maximum support, and its two-distance geometry can be prepared in $O(n + Q + K)$ time and $O(n + Q)$ auxiliary words. Discrete work uses a word-RAM with input indices, counts, widths, and intermediate index arithmetic fitting words. Metric arithmetic, including square roots, has unit cost.*

Proof. Normalize signed zeros. Complement a negative score's bit pattern and flip the sign bit of a nonnegative score, then complement the result to obtain a descending numerical key. Eight stable byte-counting passes order these 64-bit keys in $O(n)$ time. Scan complete ties and retain their starting and ending global ranks. Filter the permutation to maximum-support positions. A temporal scan with a monotone interval pointer computes nearest distances and owners in $O(Q + K)$ time. All arrays are input- or support-sized. □

Radix sorting and floating-point key transforms are established techniques [4, 7]. This assumption does not give linear sorting of arbitrary real scores or a bit-complexity bound for growing precision. Conversion defines numerical ties, including collisions between distinct large integer inputs.

The same support-event width kernel combined with this preparation gives $O(n + (L + 1)Q)$ time. This is the basic Sparse VUS evaluator; its improvement over a dense rank sweep depends on Q, rather than recall saturation. The released PR-only implementation retains $L + 1$ scalar width results, adding $O(L)$ words; streaming removes this storage. Input-sized ranking arrays remain necessary in this construction, so the memory is not support-only.

3 PRIOR ALGORITHMS AND COMPARISON SCOPE

Boniol et al.'s optimized VUS algorithms retain threshold loops within widths and support-region summation [2]. The official source additionally creates full-length prediction and label arrays per threshold. Klein removes repeated threshold-wise summation through support cumsums, but still constructs maximum-support and full-length coverage arrays at each width [8]. Benchopt cumsums over all positions and computes component-hit counts through segment maxima [1]. Table 1 distinguishes these mechanisms from the new reductions. Its costs describe inspected operations, not cross-library runtime predictions.

Sorted sweeps, complete ties and trapezoids are established ROC techniques [5]. scikit-learn 1.0.2 explicitly truncates PR at full recall after forming cumulative counts [20]. Therefore full-recall stopping and zero-area PR suffixes are not new principles. Our earlier artifact comparison includes last-original-positive stopping, which already ensures saturation. The specific questions

Input: binary labels y, finite score matrix $S \in \mathbb{R}^{M\times n}$, integer maximum width L.

1. Validate inputs; enumerate original intervals and maximum support. Prepare two distances and original-positive indicators.
2. For each model, rank complete global ties. Retain each support position's event identifier and each event's complete global prediction count.
3. For $w = 0, \dots, L$, form support weights and merged-component identifiers once. For each model, accumulate grouped positive/buffer masses and first hits; scan its events with the exact PR identity and add the width integral.
4. Divide each accumulated integral by $L+1$. Return values in input-model order.

Figure 1: Width-major shared PR evaluation. Model-specific threshold counts are never replaced by support-only counts. The algorithm stores one width's geometry, not an $(L+1)\times Q$ weight matrix.

are whether buffer mass permits an earlier stop and whether preparation avoids paying for the discarded suffix. Reversible list updates [9], union-find [23], radix ordering [4, 7] and exact accumulators [16] are established tools used in the construction.

Sylligardos reports preliminary CPU/GPU VUS acceleration [21]. The public FF-VUS code attributed to him in aVUSi [22] uses a different ramp/overlap contract; its wrapper rounds scores, and Torch can interpolate component existence across widths when memory is insufficient. This is relevant acceleration prior art, but its native runtime is not an exact-target comparison. No result here establishes superiority over an unmeasured GPU evaluator. Source pins and minimal semantic witnesses are retained with the artifact; a package name alone does not establish equality.

4 SHARED-LABEL COLLECTION EVALUATION

Consider M score vectors with the same labels and width range. Independent calls repeat original-interval extraction, support enumeration, distance preparation, and width geometry. The *geometry* variant prepares the label-only arrays once and invokes an otherwise independent support kernel for each model. The *stream* variant additionally interchanges the width and model loops, so each width's support weights and component identifiers are computed once. It still builds model-specific grouped masses and integrates all complete support events. This is ordinary work sharing over the exact representation; it does not improve the asymptotic bound per model.

Proposition 2 (Shared evaluation). *For $M \ge 1$, the shared-label PR evaluator takes $O(Mn + (L + 1)MQ)$ time under Proposition 1's model. After validation and conversion, its retained evaluation workspace is $O(MQ+n+Q+K)$. The implemented public call has the more conservative $O(Mn+MQ+n+Q+K)$ peak auxiliary-space bound for its accepted inputs.*

Proof. Label-only preparation costs $O(n + Q + K)$. Ranking and complete global counts cost $O(Mn)$ and leave two model-by-support rank/count arrays. At one width, geometry costs $O(Q+K)$

Construction	Threshold and width work	Semantic/comparison qualification
Boniol Algorithms 2/3	Per-width threshold loop; Q-support sums per threshold	Static-support reuse and cached predictions are prior techniques; the released target uses $\geq$ ties.
Klein fast-VUS	Per-width support cumsum, full-series coverage and threshold arrays	PR only; use $H = n$, not the sampled default $H = 250$. Explicit arrays cost $\Omega(Q + n + H)$ per width.
Benchopt	Per-width full-series cumsum and threshold arrays	Computes both ROC and PR; $H = n$ covers exhaustive ties. Explicit arrays cost $\Omega(n + H)$ per width.
FF-VUS	Batched width/threshold matrices and support masking	Native linear/max ramps differ from the pinned target; public wrapper rounds scores; Torch has an existence approximation fallback.
Sparse support events	$O(n + (L + 1)Q)$ with fixed-key preparation	Removes inactive thresholds while retaining global counts and exact ROC rectangles; no prefix stop is required.

Table 1: Algorithmic comparison. H is the requested threshold count; Q is maximum support. External array-cost lower bounds omit additional ramp, sorting and search work. Batched FF-VUS has no single sequential per-width loop. Our bounds use the stated arithmetic model.

and the M grouped scans cost $O(M(Q + K))$. Since $K \leq Q$, summing widths gives the time bound. Ranking needs input-sized temporary arrays; the width kernel reuses support- and interval-sized arrays. This gives the retained bound. The public NumPy finite-value check materializes an $M \times n$ Boolean temporary, and conversion of other accepted dtypes can create an $M \times n$ float64 array. Including these yields the public-call bound. □

This distinction prevents a misleading support-only memory claim. Score data already resident in float64 occupies $8Mn$ bytes independently of workspace. The stream variant's two int64 model-by-support arrays alone occupy $16MQ$ bytes; geometry-only sharing retains one model's arrays at a time. Thus sharing more work can require more live memory. We report process memory separately; NumPy validation temporaries cannot be inferred from the kernel's retained state.

Conditional alternatives. The artifact also implements reversible activation with incremental interval unions and stopping after complete-tie recall saturation. Its prefix bound is $O(n + L + \alpha(K)\sum_w J_w)$, where J_w counts the visited active positions. The stopping condition is $A + B/2 \geq P_0$ and $H = K_w$; both sides of the saturation predicate are monotone as predictions accumulate. For one original interval, a separate run aggregation implementation charges temporal runs of visited complete score classes. The detailed constructions and their versioned proofs remain in the artifact's earlier manuscript. These are conditional extensions, not the main speed claim of this paper. In particular, stopping at full recall, reversible lists, union-find and radix ordering are established tools [4, 7, 9, 20, 23].

5 INCREMENTAL RANGE PRECISION AND RECALL

The same workload analysis also exposes a different repeated-scan problem. Tatbul range precision and recall score predicted intervals rather than only points [24]. TimeSeAD [26] already maximizes the classic range F-score over every observed score threshold [3]. Its pinned implementation reconstructs predictions and interval statistics at each threshold. Thus our question is not whether exhaustive evaluation is possible, but whether those statistics require repeated full-array scans.

Target. Here predictions use the public range-F1 convention $p_i(t) = \mathbf{1}\{s_i > t\}$, distinct from the VUS convention above. Let T_j be the K true intervals and C_ℓ the J current predicted intervals. Write $m_j = |T_j \cap P|$ and r_j for the number of predicted intervals intersecting T_j. For a predicted interval C, write $u(C) = |C \cap Y|$ and $h(C)$ for the number of true intervals intersecting it. Flat positional weights and reciprocal cardinality give

$$R_\alpha = \frac{1}{K}\sum_j \left[\alpha \mathbf{1}\{m_j > 0\} + (1-\alpha)\frac{m_j}{|T_j| r_j}\right], \tag{10}$$

$$P_r = \frac{1}{J}\sum_C \frac{u(C)}{|C| h(C)}, \qquad F_1 = \frac{2P_r R_\alpha}{P_r + R_\alpha}. \tag{11}$$

Zero-overlap terms and empty outer collections contribute zero. These conventions, positional weights and cardinality are explicit; the algorithm does not cover every customization of the metric. Our primary target has $\alpha = .2$. The unchanged TimeSeAD classic comparison uses $\alpha = 0$ instead.

Proposition 3 (Local range-statistic updates). *After fixed-label preprocessing, inserting one predicted-positive position changes at most one true interval's recall statistics and at most three predicted-component terms in the precision sum. On a one-dimensional path, these changes can be maintained in constant time per insertion.*

Proof. If position i lies in true interval T_j, increment m_j and change r_j by

$$1 - \mathbf{1}\{i-1 \text{ active in } T_j\} - \mathbf{1}\{i+1 \text{ active in } T_j\}.$$

Replace only this interval's overlap term and add its existence reward on its first hit. If i is outside every true interval, it cannot connect two fragments inside one contiguous T_j, so recall statistics stay unchanged. For precision, subtract each existing adjacent component's contribution and add the merged interval's contribution. Prefix counts of true points and true interval starts recover $u(C)$ and $h(C)$ in constant time, including a true interval already in progress at the left boundary. Finally, an active neighbor of an inactive point must be a current component endpoint. Arrays storing the left boundary at each right endpoint and the right boundary at each left endpoint therefore identify both adjacent components without disjoint-set search. Update only the new outer endpoints.

Interior stale entries are never queried. Each insertion performs constant work. □

Queries and complexity. Sort scores and queries in descending order, activate every point strictly above each query, and emit a state only after all eligible ties. For H ordered queries this costs $O(n\log(n+1)+n+H)$ time and $O(n+H)$ space. Query sorting, when required, adds $O(H\log(H+1))$. With a small known query set, full score sorting is unnecessary: binary search each score among ordered queries to find its first strictly eligible query, then counting-bucket its activation. This gives $O(n\log(H+1)+n+H)$ for ordered queries, with the same extra sorting term for arbitrary query order. Points within a bucket can activate in any order because no intermediate state is returned. These are ordinary sorting and offline query techniques, not newly invented data structures.

The contribution is the metric-specific local sufficient statistics. Descending connected level sets and component trees are classical, including general-graph union-find constructions [15]. More directly, Menotti et al. give linear-time one-dimensional component-tree construction and maintain height, surface and volume attributes [14]; their presented integer-weight structure uses $O(\max(n,m))$ space for weight amplitude m. We claim neither a new component tree nor faster tree construction. Our additional state maintains label-conditioned overlap and fragmentation rewards across queried levels; the sorting term belongs to our ordered-query implementation. The endpoint simplification is elementary. Unlike the VUS reduction, this operator cannot discard off-label events. For example, with $y = (1,0,1)$, changing the prediction from $(1,0,1)$ to $(1,1,1)$ leaves recall one but changes precision from one to $1/3$. Recall itself need not be monotone under insertions because an extra fragment can increase its cardinality penalty. Neither support-only activation nor a monotone-recall shortcut is used.

Exactness and boundary conventions. The identities hold in real arithmetic for real binary64 score comparisons. Numerical sums use compensated accumulation; a length-independent rounding certificate and bitwise agreement are not claimed. The research API's experimental contract is finite real binary64 inputs; coercion of complex or large exact-integer inputs is outside that contract. The strict observed-score query set excludes the all-active state below the minimum, so its maximum is not an unrestricted optimum over all masks. The code supports a lower sentinel when that state is requested. A separate compatibility option reproduces the pinned TSB-AD all-one parser convention; ordinary interval semantics remain the mathematical default.

6 EXPERIMENTAL DESIGN

Questions and frozen decisions. We ask whether support compression helps beyond a matched compiled dense implementation, whether shared labels improve the released sparse evaluator, and whether exact computation remains worthwhile against public exact and sampled APIs. A six-series development study compared three directions: direct complete-score evaluation, shared-label computation, and integer certification followed by exact fallback. Streaming improved over released independent calls by a geometric factor of 1.193. Certificate-plus-fallback took 33.93 times as long and certified no selection early. We therefore selected streaming before generating the new evaluation scores and retained the certificate route as a negative result. The primary width is 30; width 100 is secondary and never replaces an unfavorable width-30 result.

Data and score generation. The primary protocol selects all eligible NAB, SMAP and SMD files from the public TSB-AD-U archive [12]. It excludes six named development files, source-labelled synthetic files and exact observation-plus-label duplicates within the panel. The remaining 73 complete series comprise 20 NAB, 17 SMAP and 36 SMD cases, with lengths 1,000–28,696. The raw archive SHA-256 is `0c47020d`…; the primary protocol SHA-256 is `307e0498`…. Full digests, exclusions and decoded identities are part of the artifact. No series is silently replaced after scoring.

Eight fixed CPU procedures generate Isolation Forest, LOF, nearest-neighbor, one-class SVM, PCA, KMeans, HBOS and shrinkage-Mahalanobis scores. Each uses trailing length-16 windows with repeated boundary values and an at-most-2,048 point, evenly spaced, unlabeled fit set; random procedures use seed 20261024. This is an offline transductive workload for the evaluator. In particular, LOF novelty-mode scoring includes fit points; it is not a validated detector- accuracy protocol or a reproduction of eight tuned benchmark pipelines. All 584 stored outputs retain their float64 values without rounding or jitter. Of these, 569 have more than 101 distinct scores; the range is 3–28,696 and none is constant. One KMeans fit reports five distinct clusters for the requested 16; its warning and scores are retained. Unrounded does not mean tie-free.

Reuse and dependence. Sixty-two series were measured by name in previous repository studies. New score generation does not make the recordings unseen. Moreover, the 36 SMD series contain only nine distinct int8 label sequences; 18 share a label sequence with an excluded development file. Name exclusion is therefore not geometry-disjoint evaluation. We retain the frozen analysis and add two sensitivity analyses: equal weight per identical-label block, and exclusion of the development-matching geometry. Neither grouping proves independence of physical recordings, including normalized or shifted copies. The intervals characterize this workload, not an unknown population of future domains.

Methods and unchanged public baselines. The primary PR-only experiment compares stream, geometry-only, released sparse radix, active-prefix saturation, matched dense radix, and unchanged Klein fast-VUS at `thre=n`. The dense control uses the same compiled arithmetic and nearest-interval preparation; it isolates support compression from the choice of language/compiler. After primary results, a separate protocol freezes comparison to unchanged Benchopt at `thre=n`, its matched two-output sparse-radix control, and Klein at 250 and 1,000 thresholds. Benchopt returns both ROC and PR. Its available cost to a PR consumer is useful, but only its comparison to the two-output control matches returned work. Klein is the faster measured external exact PR option on these collections. The source audit checks the current public revisions on 24 October 2026: Klein `1bae859f`, Benchopt `2b55ccb1`, official VUS `467bace0`, and aVUSi `facf4362`. FF-VUS's

different native metric and unmeasured GPU execution remain outside same-output speed ratios.

Measurement and uncertainty. All numerical libraries use one thread. Each case evaluates all eight score vectors in one complete call, including validation, preparation, sorting, allocation and numerical winner selection. Twelve method orders consist of rotations and their reversals, balancing positions. All methods receive a tiny warm-up; imports and JIT compilation are outside the warm timing. Separate collection passes directly time reading every stored file through evaluation, selection, and decision-JSON write/close. They exclude detector fitting, cold-storage latency and `fsync`. A sum of case medians is never reported as a measured collection latency.

For each series, we take the median complete-call time over the twelve trials and form a baseline/candidate log ratio. We bootstrap these paired series ratios 10,000 times with seed 20261024. The primary tables report a three- collection Bonferroni 95% interval for each comparator; this is not a joint correction over every method, width and exploratory analysis. Workflow intervals resample explicitly paired trials on the fixed collection; trials are not independent datasets. Equal-label-block sensitivity averages log ratios within blocks before resampling blocks. Raw orders and unfavorable tails remain available. Balanced positions do not guarantee removal of all carryover or shared-host effects, especially for differences of a few percent.

Correctness and selection. All exact numerical calls must agree within 10^{-11}. Numerical winner sets include every value within 10^{-10} of the maximum, preventing an arbitrary index from resolving a near tie. For an approximate winner set, regret is the best exact value minus each selected model's exact value; we report both its minimum and maximum, with 0.01 fixed as a predeclared metric-scale cutoff. Approximate errors are retained rather than treated as reasons to exclude cases. The mathematical identity, floating-point tolerance, and exact rational certificates are distinct claims. The experiment does not certify all real-arithmetic rounding errors.

7 RESULTS

Whole method and sharing. Table 2 gives the frozen width-30 result. Stream is faster than released independent calls on every one of the 73 cases, with geometric ratios 1.496, 1.271 and 1.207 across NAB, SMAP and SMD. The smallest individual ratio is 1.132. The compiled dense comparison is 3.54–3.96 times slower, so the large public-library advantage is not solely an interpreter comparison. Removing the batch increment, matched independent-call dense/released ratios are 2.368, 2.905 and 3.279, with all 73 cases favoring support compression. The maximum numerical PR discrepancy over primary and supplementary exact calls is $1.11 \cdot 10^{-15}$, with identical numerical winner sets. The matched workflow ROC comparison differs by at most $4.86 \cdot 10^{-14}$.

Table 3 measures complete collection operations. Against unchanged exact Klein, the primary workflow ratios are 22.16–23.03. Against the released sparse precursor, they are 1.169–1.332, isolating the smaller sharing increment from the whole-method gain. Stream's three collection median latencies sum to 0.382 seconds; this sum is descriptive, not a directly timed joint-corpus query.

Where the new increment comes from. Geometry-only sharing already accounts for most of the SMAP/SMD increment. Stream/geometry improvement ratios are 1.094, 1.024 and 1.025, with stream losing on 3/17 SMAP and 2/36 SMD cases at width 30. Thus the new width-major loop is a modest refinement, especially on these collections. Its strongest reason for inclusion is the explicit decomposition of repeated work; the paper's algorithmic result remains the support-event representation. At width 100, released/stream ratios are 1.413, 1.255 and 1.221. Stream loses to saturation on four cases (two NAB, one SMAP, one SMD), with worst baseline/stream ratio 0.919. The separate earlier active-prefix study likewise found that saturation failed its default-width runtime criterion in all three runs. Those losses are consistent with extra scheduling overhead when little of the support can be skipped.

Dependence sensitivity. Equal weighting of the nine SMD label blocks gives released/stream 1.217, unadjusted 95% interval [1.191,1.246], and Klein/stream 23.76 [20.68,26.59]. Removing the 18 cases whose label geometry also appears in development leaves 18 cases in eight blocks: released/stream is 1.220 [1.191,1.253]. These analyses preserve the direction of the improvement while narrowing the inference. They do not turn reused recordings into a new-domain test.

Strong public comparators and approximation. The separately frozen study corroborates Klein as faster than Benchopt's available exact API on all three collections. Comparing equal ROC+PR outputs, the sparse-radix implementation improves directly timed workflow latency by 25.3, 32.1 and 40.9 times (Table 4); all paired trial intervals exclude one. This comparison does not attribute the whole effect to the new PR batch loop.

Klein at 250 thresholds changes 14 of 73 numerical winner sets, all in SMAP or SMD; four cases have minimum and maximum regret at least 0.01. The largest regret is 0.0584. At 1,000 thresholds, three winner sets change, with no regret at least 0.01 and maximum 0.00346. NAB has zero winner changes at either resolution. Nevertheless, complete exact stream workflows are faster than 250-threshold Klein by 11.25, 9.68 and 7.27 times. On this workload, sparse exact evaluation improves both numerical fidelity and measured runtime. The result does not imply that approximation always selects a worse model, that VUS is the correct application utility, or that our detector procedures are superior.

Observed support regimes. At width 30, maximum-support fractions range from 0.0759 to 0.223 in NAB, 0.00799 to 0.140 in SMAP, and 0.00384 to 0.1148 in SMD. Original component counts range from 1 to 5, 1 to 3, and 1 to 12 respectively. These are genuinely sparse supports, the regime predicted to benefit. When $Q \approx n$, the support-sensitive asymptotic benefit disappears; extra indexing and batch storage can dominate. Short cases also make small sharing differences sensitive to allocation and interpreter overhead. Resource stress cases examine these regimes separately and do not enlarge the count of real datasets.

Collection	Cases	Released / stream	Dense / stream	Klein exact / stream
NAB	20	1.496 [1.428, 1.566]	3.543 [3.338, 3.713]	30.513 [27.399, 33.652]
SMAP	17	1.271 [1.242, 1.302]	3.693 [3.196, 4.191]	25.772 [22.941, 28.940]
SMD	36	1.207 [1.193, 1.221]	3.956 [3.736, 4.125]	25.172 [23.475, 26.695]

Table 2: Primary complete-call geometric speedups at width 30. Brackets give paired-series 95% bounds adjusted across three collections for each comparator. All include public validation, preparation, allocation and eight-model winner selection. Hardware is identified by the selected evidence receipt; per-series intervals are conditional on the fixed panel and supplemented with label-block sensitivity.

Collection	Stream	Released	Dense	Klein
NAB	0.0470	0.0626	0.1565	1.0828
SMAP	0.0528	0.0647	0.1756	1.1699
SMD	0.2824	0.3302	1.0328	6.3759

Table 3: Directly timed primary collection workflow medians, seconds. Each pass reads all score files and writes/closes its selected-model output. Detector fitting, cold storage and JIT compilation are excluded.

Does a one-expression change explain the gain? The PR inactive-event corollary admits a simple change to Klein: replace its global threshold vector by the distinct scores on maximum-width support, keeping global predicted-positive counts and every other operation unchanged. We freeze this author-created ablation after the preceding studies and before its own measurements. It reuses all 73 series and eight stored scores, with eight balanced case orders and three not-fully-balanced workflow passes. All scalar values and numerical winner sets agree; maximum PR discrepancy is $1.11 \cdot 10^{-15}$.

The patch improves Klein by 1.654, 2.274 and 2.968 times, confirming that the elementary threshold restriction explains part of the public-library gap (Table 8). The released single-model evaluator remains 12.052, 9.067 and 6.969 times faster than this patch, with all 73 case medians and each adjusted interval favoring it. For SMD, equal-label-block weighting gives 6.772 [5.835,7.253], with the same three-collection correction. The patched/stream workflow median ratios are 11.96, 10.14 and 7.46. These residual ratios include NumPy-versus-compiled implementation effects; the matched compiled dense control separately isolates support compression. The patch still performs full-length coverage/segment-max work per width and does not supply the ROC correction. Accordingly, we claim the complete exact operator representation and elimination of repeated width-dependent work, not that PR threshold restriction alone is a deep new primitive.

Compilation-matched conventional support control. A second, post-review control uses the same validation, binary64 radix ordering and complete global tie counts as our implementation, but reconstructs each width with direct sum-and-clip ramps and component support scans. It caches original-positive group mass across widths and uses support-local scratch. This is an optimized author reimplementation of the support-cumsum approach, not an unchanged public release or a literal compilation of Klein. It shares no nearest-distance geometry. Six balanced orders measure all 73 fixed cases and six complete workflow passes per collection; maximum PR discrepancy is $4.44 \cdot 10^{-16}$.

The result limits attribution (Table 9). Conventional control/released case ratios are 0.910, 0.960 and 0.978, with every adjusted interval below one and the control faster on all 73 cases. Thus this panel does not establish a separate runtime advantage for nearest geometry over an optimized compiled direct-ramp method. Control/stream ratios are 1.357, 1.201 and 1.166, and complete-workflow ratios are 1.177, 1.126 and 1.152. These remaining gains include shared preparation and width work. They are much smaller than the public NumPy-library comparisons. The direct-ramp control does not refute the exact support-event identity or the stated bound, but it precludes attributing the public-library gap to that geometric lemma alone.

A public multi-metric consumer and its bottleneck. To test an existing consumer rather than only our VUS-specific selection loop, we pin TSB-AD's unchanged `get_metrics` function at revision `303dffbf` [12]. Its scalar VUS callback receives either exact Benchopt ROC+PR or our exact radix pair; all seven other metric outputs and their upstream oracle-threshold searches remain unchanged. The recording is the largest primary input by length (SMD id 5, $n = 28{,}696$), selected before consumer outcomes. Its eight stored profiles are evaluated in four complete read-to-CSV-close passes in ABBA order, two per backend. The unchanged public autocorrelation rule selects width 125. This is a stored-profile consumer, not the upstream detector-training script or a deployed service. Source hashes are checked against the pinned revision even on cache hits.

The trace finds a strong Amdahl limit. Median VUS time falls from 0.9289 to 0.01406 seconds (66.07×), but the complete nine-metric workflow falls only from 34.260 to 33.234 seconds (1.031×). VUS accounts for 2.70–2.72% of baseline time; the remaining metric work dominates this consumer. All nine outputs agree to $3.45 \cdot 10^{-15}$. Two fixed-input repetitions per backend provide descriptive timing, not a population interval. The large VUS-specific workflow ratios therefore must not be advertised as a similar end-to-end speedup for this multi-metric benchmark.

Long recordings and controlled operating range. A follow-up frozen after the primary study and review, but before its own score generation and timings, includes every MITDB and SWaT recording in the same public archive: eight MITDB series and one SWaT series, length 419,919–650,000, with 2–199 original components. Eight names appeared in older repository studies; MITDB has seven distinct label sequences. These are additional

Collection	Paired case Benchopt / radix	Benchopt workflow (s)	Radix workflow (s)	Workflow ratio
NAB	27.284 [25.680, 28.780]	1.9961	0.0788	25.32
SMAP	34.567 [31.285, 38.479]	2.2664	0.0707	32.06
SMD	44.160 [41.109, 47.075]	14.5331	0.3551	40.92

Table 4: Supplementary matched-output comparison: both methods return exact numerical ROC and PR. Width 30, unchanged Benchopt source, same 73 series and scores. Case bounds use the same three-collection adjustment. This gain belongs to the complete sparse evaluator, not solely to the new PR batch loop.

	Klein, 250 thresholds			Klein, 1,000 thresholds		
Collection	Changed sets	Regret ≥ 0.01	Max regret	Changed sets	Max regret	250 / exact workflow
NAB	0	0	0.00000	0	0.00000	11.25
SMAP	6	3	0.05835	1	0.00345	9.68
SMD	8	1	0.05431	2	0.00283	7.27

Table 5: Approximation accuracy–cost comparison. Minimum and maximum regret threshold counts coincide in this panel; both are retained in the artifact. The final column compares directly timed PR-only workflows. Zero losses, including all NAB selections and every 1,000-threshold loss below 0.01, are retained. Values describe these fixed detector-score inputs, not detector quality.

Positive fraction	Released	Geometry	Stream	Dense
0.01	181.0	183.2	198.4	217.6
0.50	175.2	188.1	225.0	210.6

Table 6: Median OS lifetime peak RSS (MiB), three fresh processes per method, $n = 500{,}000$, $M = 8$, one centered anomaly interval, width 30. These synthetic resource cases include interpreter, libraries, inputs and warm-up; they are not measurements of auxiliary space or additional real datasets.

fixed workloads, not nine independent unseen populations. All use the same eight detector procedures, width 30 and seven cyclic method orders. Separate collection workflows use only three orders and are not fully position-balanced. Exact-output PR and ROC discrepancies are at most $2.56 \cdot 10^{-15}$ and $8.62 \cdot 10^{-14}$, respectively.

MITDB and SWaT workflow median ratios for Klein/stream are 23.10 and 17.70; MITDB's latency drops from 53.14 to 2.30 seconds (Table 7). Matched ROC+PR ratios are 29.50 and 25.66. The additional sharing gain is less decisive: released/stream workflow median ratios are 1.220 and 1.086, but paired-geometric trial intervals [0.925,1.340] and [0.955,1.165] include one. Stream also loses to geometry-only sharing on one MITDB case. Thus long-recording evidence supports the whole evaluator more strongly than the small batch increment.

Sixteen deduplicated synthetic conditions vary one axis from a common $n = 100{,}000$, $M = 8$, positive fraction 0.01, $K = 1$, width-30 anchor: n up to one million, positive fraction up to 0.9, K up to 1,000, M up to 32, width up to 300, and a 16-level tied-score condition. Four cyclic orders balance method positions. These cases test evaluator cost, not detector accuracy. The artifact retains their paired trial geometric ratios, unadjusted conditional 95% intervals and operating-range figure. At positive fraction 0.9, matched dense/released is 0.980 [0.962,0.9994]: support compression no longer improves runtime (the ratio of medians is 0.995). At width 0 its ratio is 1.065, versus 16.36 at width 300. With $M = 1$, released/stream is only 1.023; sharing has little work to amortize. These observations delimit the contribution rather than imply universal dominance. Increasing K also increases expanded support and is not an isolated intervention on component overhead.

Post-timing diagnostics separately instrument the earlier active-prefix algorithm. A dense independent reconstruction of interval ramps, complete score ties and saturation verifies every visited-count array. At the one-million-point random-score case, this algorithm visits 99.94% of active positions, explaining why recall saturation is not a general substitute for support compression. At $K = 1{,}000$, it visits 33.79%; these counts do not measure the different stream implementation's work or establish its timing.

Original VUS execution on a second CPU.. The original one-command VUS artifact was executed from a fresh checkout on hosted Linux with an AMD EPYC 7763, regenerating all scores from pinned raw data and sources. Both record audits passed. At width 30, released/stream case ratios were 1.886, 1.580 and 1.340; three-collection-adjusted 95% intervals were [1.732,2.043], [1.526,1.644] and [1.322,1.359]. All 73 case medians favored stream. Klein workflow ratios were 20.06/20.28/16.17; matched ROC+PR Benchopt/radix workflow ratios were 22.38/30.40/42.55. Hosted-runner load is uncontrolled: a NAB stream workflow of 0.111 seconds versus its 0.061-second median is retained. Hosts are not pooled.

Generated scores and approximation outcomes differ across platforms. The 250-query VUS approximation changes 13 winner sets with three losses of at least .01 on Linux, versus 14 and four on the Mac. A separate diagnostic archives fresh two-host score arrays, standardized inputs, fitted SVM states and backend metadata. Identical raw and standardized inputs yield 104 versus 105 support vectors; both score arrays reproduce their metric values on one

Collection	Series	Stream (s)	Klein (s)	Released (s)	Paired geometric Klein/stream [95% CI]	Matched pair ratio
MITDB	8	2.300	53.141	2.806	22.10 [19.00, 25.00]	29.50
SWaT	1	0.245	4.339	0.266	17.62 [16.37, 18.90]	25.66

Table 7: Separately frozen long-recording follow-up, $M = 8$, width 30, Apple M1 Pro. Times are medians of three complete collection passes. PR intervals resample paired timing trials on fixed inputs and are unadjusted; three workflow orders are not fully position-balanced. The matched-pair column is the ratio of median Benchopt and radix workflow times for ROC+PR. SWaT is one recording, not an estimate over a population of independent series.

Collection	Klein / patched Klein	Patched Klein / released	Patched Klein / stream	Workflow patched/stream
NAB	1.654 [1.496, 1.842]	12.052 [10.325, 13.812]	17.956 [14.605, 21.626]	11.96
SMAP	2.274 [2.149, 2.401]	9.067 [8.179, 10.052]	11.534 [10.190, 12.994]	10.14
SMD	2.968 [2.842, 3.089]	6.969 [6.586, 7.270]	8.396 [7.891, 8.794]	7.46

Table 8: Author-created minimal ablation: replace only Klein's threshold list with all distinct maximum-support scores; retain global counts and other code. Eight balanced case orders on the original 73 series, width 30, Apple M1 Pro; case intervals are 95% bounds adjusted across three collections per comparison. The final column uses three not-fully-balanced workflow passes. Remaining gains include implementation differences; the compiled dense control separately isolates support compression. The patched code is not an upstream release.

Collection	Control / released	Control / stream	Released / stream	Workflow control/stream
NAB	0.910 [0.905, 0.915]	1.357 [1.291, 1.427]	1.492 [1.419, 1.567]	1.177
SMAP	0.960 [0.945, 0.973]	1.201 [1.180, 1.224]	1.251 [1.218, 1.287]	1.126
SMD	0.978 [0.969, 0.985]	1.166 [1.158, 1.174]	1.193 [1.181, 1.205]	1.152

Table 9: New author-created compiled conventional support-cumsum control, with matched validation, radix ordering and global counts, direct per-width ramps and cached original-positive mass. Six balanced orders for all 73 primary cases and six balanced workflow passes, width 30, Apple M1 Pro. Brackets are three-collection-adjusted 95% intervals. Ratios below one favor the control; all scalar cases favor it over released. This is not unchanged public Klein.

evaluator host and agree with unchanged Klein within $5.56 \cdot 10^{-17}$. This localizes the difference after standardization without identifying a specific instruction or compiler cause. The diagnostic Linux arrays are not recovered original-run arrays: their hashes differ and other metric values change by up to .000583. The 1,000-query counts remain three changed winner sets and no losses of .01. The artifact retains full diagnostic values and provenance; fixed-input exactness is distinct from cross-host score generation.

Process memory. A separately frozen resource study selects the real cases with largest length, support fraction and component count, repeats the largest case's eight scores four times for a declared $M = 32$ stress case, and adds two synthetic density cases at $n = 500{,}000$, $M = 8$. Three fresh processes per method record OS lifetime peak RSS, including interpreter, libraries, inputs and tiny JIT warm-up. This measures process resource use; subtracting two high-water marks does not measure exact auxiliary space. On the real $M = 8$ cases, stream peaks at 113.9–119.0 MiB; on the repeated $M = 32$ case, 127.4 MiB versus released 125.5 MiB. The two synthetic cases expose the tradeoff more clearly (Table 6). At positive fraction 0.5, stream's peak is 225.0 MiB versus 175.2 MiB for released and 210.6 MiB for dense. Thus the batch speed claim does not imply lowest peak memory. The largest numerical PR discrepancy in this resource study is $1.67 \cdot 10^{-16}$; no timing conclusion or real-data generalization is drawn from its synthetic inputs.

8 RANGE-METRIC GENERALIZATION EXPERIMENT

This is a separately frozen exploratory follow-up motivated by the small total gain of the VUS-only public consumer. It reuses all 73 recordings and 584 fixed profiles, not a held-out dataset. No old VUS outcome is relabeled as new range evidence. Four complete calls are compared at the 100 evenly spaced strict thresholds used by the public range-F1 consumer: vectorized prefix/interval recomputation, a fused compiled raster scan, the score-sorted endpoint sweep, and query bucketing. The raster uses constant reducer workspace and no per-threshold interval arrays; only methods requiring prefix preparation pay for it. Thus Python-versus-compiled overhead does not explain the main comparison. Eight cyclic/reverse orders balance four grid methods. A separate AB/BA study compares raster and sweep over every observed distinct threshold, with two complete eight-profile calls per method. Timings include validation, query construction, preparation, sorting, curves and maximizing-value selection; they exclude training and file I/O.

Table 10: Range-F1 study: case geometric speedup over the fused compiled raster, with familywise 95% intervals over three collections separately per comparison. Apple rows are the original M1 Pro run; Linux rows are the fresh combined EPYC 7763 run. Hosts and query contracts are not pooled.

Host / collection	Grid100 raster/bucket	Distinct raster/sweep	Distinct wins/cases
Apple NAB	1.719 [1.625, 1.814]	27.80 [18.74, 41.94]	20/20
Apple SMAP	1.994 [1.817, 2.252]	21.44 [8.57, 43.14]	16/17
Apple SMD	1.969 [1.884, 2.095]	128.08 [95.09, 154.61]	36/36
Linux NAB	1.900 [1.616, 2.239]	29.35 [18.81, 46.08]	20/20
Linux SMAP	2.898 [2.624, 3.260]	27.76 [11.63, 53.75]	16/17
Linux SMD	3.574 [3.412, 3.789]	163.60 [125.92, 193.02]	36/36

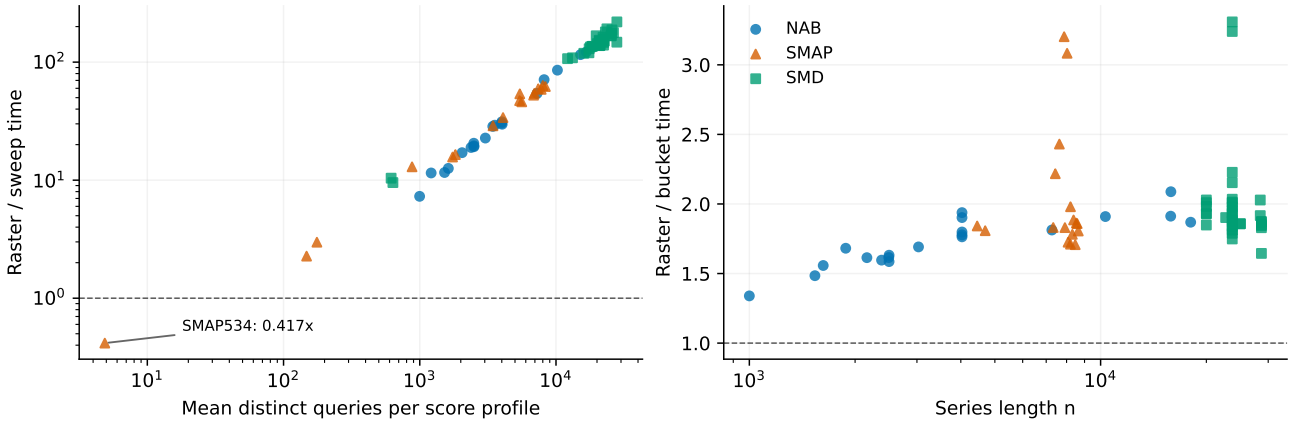


Figure 2: All 73 original Apple range-metric cases, each with eight score profiles. Ratios use median complete-call times against the fused compiled raster. Left: exhaustive observed thresholds; right: the fixed 100-query contract. The low-cardinality loss remains visible. These are descriptive fixed-panel observations, not controlled interventions on query count or series length.

In the original Apple study, every canonical precision/recall/F1 curve is archived for every method and profile; each repetition's output hash must match. A separate auditor imports no metric implementation and reconstructs curve agreement, maximizing values, thresholds, winner sets and paired intervals. Maximum reconstructed curve discrepancy is $3.00 \cdot 10^{-15}$, with no cross-method model-winner-set differences. This is numerical cross-implementation evidence, not an independent mathematical oracle on every real input. Set-intersection oracles additionally cover exhaustive short tied inputs and randomized boundary/query cases.

Table 10 shows a clear measured algorithmic advantage over the fused raster at exhaustive observed thresholds; Figure 2 retains every case. For the 100-grid, bucketing wins on all 73 case medians; the score-sorted sweep is 1.39/1.41/1.87 times slower than bucketing geometrically. It wins on one SMAP case, which remains reported. The exhaustive sweep loses on SMAP534, whose eight profiles have only 3–11 distinct scores: raster takes 0.838 ms versus 2.012 ms for sweep. Small distinct cardinality can favor scanning. For SMD, equal weighting of its nine label blocks gives raster/sweep 112.34 [46.21, 167.79] and raster/bucket 2.028 [1.869, 2.435]; this does not establish independent physical sources.

What exact threshold evaluation changes. Holding scores, labels and $\alpha = .2$ fixed, replacing the 100-grid by observed distinct thresholds changes 24/73 numerical winner sets. Fifteen have no overlapping winning model; twelve incur at least .01 range-F1 regret when the best grid-winning model is evaluated at its exhaustive optimum. Per-collection nonoverlap counts are 6/20, 3/17 and 6/36; maximum regrets are .439, .150 and .127. All ties use the predeclared 10^{-10} policy. These are oracle evaluation and selection effects within this reused eight-profile panel, not improved detector accuracy or a threshold that can be deployed without labels. Rank changes from metric approximation are not a new conceptual discovery; the new result is the exact update algorithm and its measured cost/fidelity tradeoff. In a post-review descriptive comparison of the same recorded case medians, exhaustive sweep still costs 2.19/2.14/3.01 times our grid100 bucket evaluator on NAB/SMAP/SMD. The query contracts differ: this is the price of exhaustive coverage relative to the fastest measured grid method, not a same-output speed comparison or a prespecified endpoint.

Fresh combined reproduction. One command regenerated scores and executed the VUS primary/supplement and all three range studies on an AMD EPYC 7763 Linux host. The frozen source and design checks, four independent record audits and selected-cut/stress validation passed. Complete generated arrays and

curves are retained. The range result again has maximum cross-method curve discrepancy $3.00 \cdot 10^{-15}$ and no cross-method winner disagreement. Bucketing wins all 73 case medians; SMAP534 remains the sole exhaustive-sweep loss (raster/sweep 0.661). SMD equal-label-block raster/sweep is 146.45 [67.70, 208.30]. All 73 generated score matrices differ bitwise from the Mac; differences are not uniformly small. Aggregate selection counts remain 24/15/12, but SMD188 and SMD200 exchange winner-change and nonoverlap status; maximum SMD regret is .0797 instead of .127. These are regenerated-input results, not an isolated CPU intervention. The same two public SMD187 calls are censored; completed public scalar tolerances and million-point stress checks pass. A separate range-only EPYC 9V45 execution also supports the direction and is archived, but the combined run was selected before inspecting either timing report; favorable hosts were not selected or pooled.

Public comparison and application boundary. Original Apple public comparisons are a separate length-selected panel: the three shortest recordings in each collection, all eight profiles. TSB-AD range-F1 uses its 100-grid and $\alpha = .2$; TimeSeAD classic uses observed strict thresholds and $\alpha = 0$, with float32 precision/recall/F1 storage. The latter requires a distinct numerical tolerance and is not silently treated as the primary target. Each public call has a prospectively declared 60-second budget; timeouts remain censored observations, not exact latencies or parity successes. Grid calls match the public scalar within $6.67 \cdot 10^{-16}$; completed TimeSeAD calls match within $5.66 \cdot 10^{-8}$. Descriptive public-grid speedups are 501/272/384 geometrically on the three length-selected panels. TimeSeAD ratios are 402/474 for NAB/SMAP; SMD181 and SMD185 take 31.96 and 37.71 seconds versus 21.68 and 25.14 milliseconds. Both public calls for SMD187 exceed the 60-second limit; candidate calls have a 33.65-ms median, but no completed public-output check is claimed for that recording. These public-library factors combine algorithm and runtime differences. The fused compiled comparison in Table 10 is the evidence for an algorithmic gain.

A separate check applies unchanged TSB-AD binary-prediction range-F1 to each of the 584 profiles at both recorded maximizing cuts. All 1,168 values agree within $4.62 \cdot 10^{-16}$; this validates the value at each chosen cut, not independent maximality over untested cuts. Three one-million-point numerical stress inputs (alternating labels, one interval and dense tied scores), each with 257 queries, compare all four methods. Maximum curve error is $2.23 \cdot 10^{-12}$, below the predeclared 10^{-9} stress tolerance. This is empirical numerical evidence, not a proof for arbitrary sequence lengths. These operator studies do not measure a new whole-nine-metric acceleration. Affiliation, the largest measured stage of the public consumer, remains unchanged. The existing 1.031-fold whole-consumer result therefore remains the only measured application-wide result in this manuscript.

9 RELATED WORK AND NOVELTY BOUNDARY

The closest algorithmic work is the original VUS operator and its optimized variants [2, 18]. We preserve that operator. Temporal weighting and proximity-aware PR are also established by PATE [6]; we neither introduce those ideas nor compare a different metric's runtime as if it returned VUS. Sylligardos's efficient metric work [21] and current public implementations motivate the semantic and source audit in Section 3.

TSB-AD [12] studies large detector benchmarks and metric reliability; TAB [19] unifies benchmark data, methods and evaluation. Our operator is a possible component of those systems, not a replacement benchmark. TSB-AutoAD [11] compares selection, ensembling and generation; RAMSAD [17] retrieves historical detector profiles for selection. We reduce the cost of computing labeled profiles, without claiming superior unlabeled selection. mTSBench [30] covers 344 multivariate series and 24 detectors, broader than our targeted univariate operator workload. Analyses of foundation-model pipelines [31] likewise address detection methodology rather than the exhaustive scalar representation studied here.

Recent formal analysis of 37 metrics explicitly excludes score-based volume-under-surface metrics from its formal scope [27]. The resulting LARM/ALARM axioms do not establish our computational identity. CCE [29] gives a different linear-time metric; its speed is not an equal-output baseline. The metric taxonomy [28] concerns application requirements. Adversarial seed-shopping analysis [13] varies reporting choices and compares metrics, while our approximation experiment holds labels, score vectors and the target fixed. None of these distinctions makes rank changes themselves a new discovery. For VUS, the narrow contribution is the exact event reduction, complete global counts and dynamic integral correction. For range precision/recall, Tatbul's metric and TimeSeAD's exhaustive observed-threshold evaluation already exist. Classical component-tree methods already process connected level sets, including linear-time one-dimensional construction [14]. Our additional result is the local overlap/cardinality update and its measured advantage over a fused same-compiler raster. The old VUS software precursor does not contain this range-metric generalization.

10 REPRODUCIBILITY AND SCOPE

The artifact binds the public archive, each raw member, decoded observations, labels, scores, method sources, trial orders and output records by SHA-256. A single command creates a new isolated workspace, downloads pinned sources, generates scores, executes the VUS primary/supplement and range studies, and reconstructs their numerical tables as JSON (TeX rendering and manuscript compilation are separate):

```
python experiments/reproduce_range_sweep.py
  --include-vus --output /tmp/exact-replica
```

It refuses an existing destination. Independent reconstruction imports no metric producer and checks complete coverage, input/protocol/run bindings, scalar values, winner sets, paired intervals and every principal-table field. Corruption tests exercise changed intervals, fabricated summary entries, replaced inputs, duplicate cases and false regret. Historical audit revisions remain in the artifact. Fresh execution on the second CPU additionally checks the full pipeline beyond reconstruction of recorded outputs.

The recorded environment is Python 3.12.4, NumPy 1.26.4, Numba 0.59.1, pandas 3.0.1, scikit-learn 1.8.0 and SciPy 1.17.1. The public consumer uses a separate statsmodels 0.14.6 overlay; an earlier 0.14.2/pandas import failure is retained and occurred before any measurements. Detector scores can differ across platforms even with these package pins; the diagnosed SVM fitted states differ despite identical standardized inputs. Each execution workspace retains its CPU, platform, package versions and independently generated arrays. The original VUS-only Linux archive omitted generated NPZ arrays; a later two-host diagnostic pair is archived separately. The new combined execution retains all generated NPZ arrays and complete range curves. We do not pool hosts into a single interval. Our CPU measurements do not establish GPU superiority; the literature comparison is a bounded search of the stated sources.

The completed combined one-command execution covers VUS primary/supplement and range main/public/validation studies on freshly generated inputs. Its archived reports were independently reconstructed after download, with exactly equal JSON objects on the auditing host. The original and auxiliary runs remain identified separately. Long-recording/controlled, resource, minimal-Klein and compiled conventional-support and public-consumer studies have separate frozen protocols and commands in the artifact instructions. The operating-range wrapper regenerates its inputs and checks its full design before timing; its stages were executed in the recorded run, while the wrapper itself has not received a second complete fresh execution. This boundary is separate from the completed fresh primary/supplement run on Linux.

Limits. The principal evaluation uses short-to-moderate univariate series with sparse anomalies and shared labels; the separate follow-up reaches 650,000 points on real data and one million synthetically. Dense support and numerous fragments are covered by controlled cases, not broad real-domain samples. Different component-merging conventions, other VUS ramps, cold compilation and training-dominated pipelines remain separate regimes. The conventional compiled control is faster than our released single-model method at the primary width, so separate geometric superiority is not claimed. Whole-operator speed can matter for repeated stored-profile queries even when the absolute savings on one small collection are seconds. That use case must not be generalized to end-to-end detector training. The mathematical statements use unit-cost metric arithmetic; finite-precision experiments provide numerical parity, not an all-input formal rounding proof. Batching uses more retained support state, and its full public peak includes model-by-time validation/conversion arrays.

11 CONCLUSION

For the original dynamic VUS operator, inactive thresholds can be removed from an exhaustive scalar computation only with complete global tie counts and the appropriate ROC predecessor term. The resulting support-sensitive evaluator preserves the target and permits shared-label evaluation. Three public collections show numerical agreement and lower complete-call and workflow cost than the measured exact comparators; the advantage also holds against the measured sampled API. Modest incremental sharing gains, unfavorable prefix cases, dependence and memory costs define the supported scope. A separate endpoint representation maintains range precision/recall through component merges, removing repeated raster scans at exhaustive observed thresholds. Its strongest measured benefit is against a fused compiled conventional scan; low distinct cardinality remains a loss regime. These operator improvements do not imply a new metric, unlabeled selector or universal application speedup.

REFERENCES

[1] Benchopt contributors. 2026. Time-series foundation-model benchmark: VUS metrics. https://github.com/benchopt/benchmark_tsfm Software, audited revision 2b55ccb1, 29 September 2026.

[2] Paul Boniol, Ashwin K. Krishna, Marine Bruel, Qinghua Liu, Mingyi Huang, Themis Palpanas, Ruey S. Tsay, Aaron Elmore, Michael J. Franklin, and John Paparrizos. 2025. VUS: Effective and Efficient Accuracy Measures for Time-Series Anomaly Detection. *The VLDB Journal* 34, Article 32 (2025), 26 pages. https://doi.org/10.1007/s00778-025-00907-x

[3] TimeSeAD contributors. 2025. TimeSeAD classic range F-score evaluator. https://github.com/wagner-d/TimeSeAD/blob/bb8efa9ce17f9cfb0bab77df08c32d986de8429e/timesead/evaluation/evaluator.py Source revision bb8efa9ce17f9cfb0bab77df08c32d986de8429e, inspected 24 October 2026.

[4] Erik Demaine. 2011. Lecture 7: Linear-Time Sorting. MIT OpenCourseWare, 6.006 Introduction to Algorithms, Fall 2011. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/bf7d79105762bf79bbc0925438e1468a_MIT6_006F11_lec07.pdf Accessed 2026-10-21.

[5] Tom Fawcett. 2006. An Introduction to ROC Analysis. *Pattern Recognition Letters* 27, 8 (2006), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010

[6] Ramin Ghorbani, Marcel J. T. Reinders, and David M. J. Tax. 2024. PATE: Proximity-Aware Time series anomaly Evaluation. arXiv:2405.12096 [cs.LG] https://arxiv.org/abs/2405.12096

[7] Michael Herf. 2001. Radix Tricks. https://stereopsis.com/radix.html Accessed 2026-10-21.

[8] Tom Louis Klein. 2026. fast-VUS. https://github.com/tomlouisklein/fast-VUS Software, first commit 12 October 2026; audited revision 1bae859f, 16 October 2026.

[9] Donald E. Knuth. 2000. Dancing Links. arXiv:cs/0011047 https://arxiv.org/abs/cs/0011047

[10] Youngmin Ko. 2026. Sparse VUS, version 0.3.0. https://github.com/mandu5/sparse-vus/releases/tag/v0.3.0 Public software and working-manuscript precursor, released 21 October 2026; source revision 68d6e6de627a734880e92ecc94d3ffc9eb38a89d.

[11] Qinghua Liu, Seunghak Lee, and John Paparrizos. 2025. TSB-AutoAD: Towards Automated Solutions for Time-Series Anomaly Detection. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4364–4379. https://doi.org/10.14778/3749646.3749699

[12] Qinghua Liu and John Paparrizos. 2024. The Elephant in the Room: Towards a Reliable Time-Series Anomaly Detection Benchmark. In *Advances in Neural Information Processing Systems*, Vol. 37. 108231–108261. https://github.com/TheDatumOrg/TSB-AD

[13] Zongye Lyu. 2026. Did We Actually Fix It? An Independent Adversarial Stress-Test of Post-Point-Adjustment Evaluation Metrics for Time-Series Anomaly Detection. arXiv:2607.11969 [stat.ML] https://arxiv.org/abs/2607.11969

[14] David Menotti, Laurent Najman, and Arnaldo de Albuquerque Araújo. 2007. 1D Component Tree in Linear Time and Space and Its Application to Gray-Level Image Multithresholding. In *Proceedings of the 8th International Symposium on Mathematical Morphology*. INPE, 437–448. https://hal.science/hal-00622373

[15] Laurent Najman and Michel Couprie. 2006. Building the Component Tree in Quasi-Linear Time. *IEEE Transactions on Image Processing* 15, 11 (2006), 3531–3539. https://doi.org/10.1109/TIP.2006.877518

[16] Radford M. Neal. 2015. Fast Exact Summation Using Small and Large Superaccumulators. arXiv:1505.05571 [cs.NA] https://arxiv.org/abs/1505.05571 Version 1, 21 May 2015.

[17] Christos Panourgias, Roberto Stanzione, Adrien Petralia, Themis Palpanas, and Paul Boniol. 2026. Retrieval Over Training: Similarity search-based Model Selection for Time Series Anomaly Detection. arXiv:2610.20243 [cs.LG] https://arxiv.org/abs/2610.20243

[18] John Paparrizos, Paul Boniol, Themis Palpanas, Ruey S. Tsay, Aaron Elmore, and Michael J. Franklin. 2022. Volume Under the Surface: A New Accuracy Evaluation Measure for Time-Series Anomaly Detection. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2774–2787. https://doi.org/10.14778/3551793.3551830

[19] Xiangfei Qiu, Zhe Li, Wanghui Qiu, Shiyan Hu, Lekui Zhou, Xingjian Wu, Zhengyu Li, Chenjuan Guo, Aoying Zhou, Zhenli Sheng, Jilin Hu, Christian S.

Jensen, and Bin Yang. 2025. TAB: Unified Benchmarking of Time Series Anomaly Detection Methods. arXiv:2506.18046 [cs.LG] https://arxiv.org/abs/2506.18046

[20] scikit-learn developers. 2021. Precision–Recall Curve, version 1.0.2. https://github.com/scikit-learn/scikit-learn/blob/1.0.2/sklearn/metrics/_ranking.py Full-recall truncation in precision_recall_curve; inspected 2026-10-21.

[21] Emmanouil Sylligardos. 2025. Fast or Accurate? Rethinking Time Series Anomaly Detection. In *VLDB Ph.D. Workshop*. https://www.vldb.org/2025/Workshops/VLDB-Workshops-2025/PhD/PhD25_17.pdf Section 4.

[22] Emmanouil Sylligardos. 2026. FF-VUS implementation in aVUSi. https://github.com/doanthihoaithu/aVUSi Software, audited revision facf4362, 23 June 2026.

[23] Robert E. Tarjan and Jan van Leeuwen. 1984. Worst-Case Analysis of Set Union Algorithms. *J. ACM* 31, 2 (1984), 245–281. https://doi.org/10.1145/62.2160

[24] Nesime Tatbul, Tae Jun Lee, Stan Zdonik, Mejbah Alam, and Justin Gottschlich. 2018. Precision and Recall for Time Series. In *Advances in Neural Information Processing Systems*. https://papers.neurips.cc/paper_files/paper/2018/hash/8f468c873a32bb0619eaeb2050ba45d1-Abstract.html

[25] TheDatumOrg. 2026. VUS: Official Metric Implementation. https://github.com/TheDatumOrg/VUS/blob/467bace0c103f5c2bdb4900306d82939bcac8ff9/vus/utils/metrics.py Pinned revision 467bace0; RangeAUC_volume_opt with exhaustive thresholds, inspected 2026-10-21.

[26] Dennis Wagner, Tobias Michels, Florian C. F. Schulz, Arjun Nair, Maja Rudolph, and Marius Kloft. 2023. TimeSeAD: Benchmarking Deep Multivariate Time-Series Anomaly Detection. *Transactions on Machine Learning Research* (2023). https://openreview.net/forum?id=iMmsCI0JsS

[27] Dennis Wagner, Arjun Nair, Billy Joe Franks, Justus Arweiler, Aparna Muraleedharan, Indra Jungjohann, Fabian Hartung, Mayank C. Ahuja, Andriy Balinskyy, Saurabh Varshneya, Nabeel Hussain Syed, Mayank Nagda, Phillip Liznerski, Steffen Reithermann, Maja Rudolph, Sebastian Vollmer, Ralf Schulz, Torsten Katz, Stephan Mandt, Michael Bortz, Heike Leitte, Daniel Neider, Jakob Burger, Fabian Jirasek, Hans Hasse, Sophie Fellenz, and Marius Kloft. 2025. Formally Exploring Time-Series Anomaly Detection Evaluation Metrics. arXiv:2510.17562 [cs.LG] https://arxiv.org/abs/2510.17562

[28] Kaixiang Yang, Jiarong Liu, Yupeng Song, Shuanghua Yang, and Yujue Zhou. 2026. A Problem-Oriented Taxonomy of Evaluation Metrics for Time Series Anomaly Detection. arXiv:2511.18739 [cs.AI] https://arxiv.org/abs/2511.18739

[29] Zhijie Zhong, Zhiwen Yu, Yiu ming Cheung, and Kaixiang Yang. 2025. CCE: Confidence-Consistency Evaluation for Time Series Anomaly Detection. arXiv:2509.01098 [cs.LG] https://arxiv.org/abs/2509.01098

[30] Xiaona Zhou, Constantin Brif, and Ismini Lourentzou. 2026. mTSBench: Benchmarking Multivariate Time Series Anomaly Detection and Model Selection at Scale. *Transactions on Machine Learning Research* (2026). https://openreview.net/forum?id=8LfB8HD1WU Code revision 0ebf90da; dataset revision 8dea9af6.

[31] Xiaokun Zhu, Louis Carpentier, and Mathias Verbeke. 2026. When Foundation Models are One-Liners: Limitations and Future Directions for Time Series Anomaly Detection. In *International Conference on Learning Representations*. https://openreview.net/forum?id=H27kvyG4qf Released scores: KU Leuven RDR, V1, doi:10.48804/MCVI4N.